

Санкт-Петербургский государственный университет

ЛАНОВАЯ Александра Юрьевна

Выпускная квалификационная работа

**Разработка методов валидации и
обнаружения условных зависимостей
включений в табличных данных**

Уровень образования: бакалавриат

Направление *09.03.04 «Программная инженерия»*

Основная образовательная программа *СВ.5080.2022 «Программная инженерия»*

Научный руководитель:
доцент кафедры ИАС, к. ф.-м. н., Е. Г. Михайлова

Консультант:
ассистент кафедры ИАС, Г. А. Чернышев

Рецензент:
ведущий эксперт, ООО «СИГМА», Д. Ю. Гончаров

Санкт-Петербург
2026

Saint Petersburg State University

Alexandra Lanovaya

Bachelor's Thesis

Development of methods for validating and detecting conditional inclusion dependencies in tabular data

Education level: bachelor

Speciality *09.03.04 «Software Engineering»*

Programme *CB.5080.2022 «Software Engineering»*

Scientific supervisor:
docent of IAS department E.G. Mikhailova

Consultant:
assistant professor of IAS department G.A. Chernishev

Reviewer:
lead expert "SIGMA" D.Y. Goncharov

Saint Petersburg
2026

Оглавление

Введение	4
1. Постановка задачи	7
2. Обзор	8
2.1. Условные зависимости включения	8
2.2. Алгоритмы поиска AIND	16
2.3. Алгоритмы поиска условий CIND	17
2.4. Архитектура Desbordante	25
3. Описание решения	27
3.1. Иерархия классов	27
3.2. Реализация алгоритма Cure	28
3.3. Алгоритм валидации	30
3.4. Интеграция в Desbordante	31
4. Эксперимент	34
4.1. Набор данных и оборудование	34
4.2. Эквивалентность выдачи оптимизированных алгоритмов	36
4.3. Влияние оптимизаций на производительность	36
4.4. Сравнение алгоритмов поиска условий	37
4.5. Производительность валидатора	41
Заключение	43
Список литературы	44

Введение

Реляционная модель данных, предложенная Коддом [9], остаётся доминирующей парадигмой организации информации. Несмотря на распространение альтернативных подходов [14], реляционные СУБД сохраняют ведущие позиции в индустрии благодаря зрелой экосистеме и развитым механизмам обеспечения целостности [2].

Однако с ростом объёмов данных схемы баз данных неизбежно деградируют: ограничения устаревают, а реальные закономерности перестают соответствовать первоначально заданным правилам [1]. В связи с этим возрастает потребность в профилировании данных — систематическом анализе содержимого баз данных для извлечения метаданных [26, 12]. Результаты профилирования применяются при оптимизации запросов [7], обнаружении аномалий и повышении качества данных [29] по некоторым заранее выбранным метрикам.

Основным инструментом профилирования служат зависимости данных — формально описанные закономерности в хранимой информации [17]. К ним относятся, в частности, зависимости включения (Inclusion Dependencies, IND) [6, 15]: отношение между атрибутами таблиц, при котором множество значений одного атрибута является подмножеством значений другого. Зависимости включения применяются при обнаружении внешних ключей [25] и поддержании ссылочной целостности [24].

Автоматический поиск зависимостей включения является NP-трудной задачей [19], поэтому для её решения был разработан ряд специализированных алгоритмов: SPIDER [21], BINDER [20], MIND [15] и FAIDA [22]. Тем не менее на практике зависимости включения далеко не всегда выполняются на всех данных. Для таких случаев были введены приближённые зависимости включения (Approximate IND, AIND) [22], допускающие определённую долю нарушений. Однако приближённые зависимости дают лишь агрегированную оценку доли нарушений, не раскрывая того, на каком именно подмножестве данных зависимость выполняется.

Эту проблему решают условные зависимости включения (Conditional

Inclusion Dependencies, CIND), предложенные Браво и Фанем [5]. CIND расширяют классические IND добавлением таблицы условий — набора шаблонов, определяющих подмножество кортежей, на котором зависимость должна выполняться. Такой подход позволяет выявлять нарушения ссылочной целостности на определённых срезах данных [10, 18], оптимизировать выполнение запросов [27], а также обнаруживать закономерности, проявляющиеся только при определённых условиях [8].

Для поиска условий CIND в [18] были предложены два алгоритма: CINDERELLA, основанный на ассоциативных правилах с использованием адаптированного алгоритма Apriori [3], и PLI-CIND, работающий с пересечением позиционных списков атрибутов. CINDERELLA осуществляет поиск кандидатов в ширину, обеспечивая высокую скорость ценой повышенного потребления памяти, тогда как PLI-CIND использует поиск в глубину с существенно меньшими требованиями к памяти.

Альтернативный подход к поиску условий CIND был предложен О. Кюре [11] в контексте повышения качества фармацевтических баз данных. Алгоритм Cure использует двухфазную схему: на первой фазе для каждой AIND перебираются пары атрибутов условия и для каждой пары находятся совместно встречающиеся значения с поддержкой не ниже заданного порога; на второй фазе полученные шаблоны объединяются в минимальное покрытие, причём при совпадении значений левой части допускается дизъюнкция значений в правой. В отличие от CINDERELLA и PLI-CIND, алгоритм управляется единственным параметром — минимальной поддержкой.

Алгоритмы профилирования данных реализованы в нескольких инструментах, наиболее известными из которых являются Metanome [13] и Desbordante [16]. Desbordante — высокопроизводительный инструмент с открытым исходным кодом, написанный на C++ и предоставляющий интерфейсы через Python и командную строку.

Ранее в рамках бакалаврской работы [31] была выполнена первоначальная реализация алгоритмов CINDERELLA и PLI-CIND¹. Однако

¹<https://github.com/Desbordante/desbordante-core/pull/559>

эта реализация имеет ряд ограничений: алгоритмы не были оптимизированы по производительности, задача валидации — проверки того, выполняется ли заданная CIND на конкретных данных, — не рассматривалась, а альтернативные алгоритмы поиска условий не были реализованы. Каждое из этих ограничений снижает практическую ценность реализации: оптимизация необходима для применимости к большим наборам данных, валидация — для проверки гипотез без запуска полного поиска [28, 4], а наличие альтернативных алгоритмов — для выбора подхода под конкретную задачу.

Данная работа посвящена оптимизации существующих алгоритмов поиска условных зависимостей включения, разработке и реализации алгоритма валидации CIND, а также реализации альтернативного алгоритма поиска CIND в инструменте профилирования данных Desbordante.

1. Постановка задачи

Целью работы является оптимизация алгоритмов поиска условных зависимостей включения, разработка алгоритма валидации CIND, реализация альтернативного алгоритма поиска CIND и их интеграция в инструмент профилирования данных Desbordante с последующим экспериментальным исследованием.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Провести оптимизацию существующих реализаций алгоритмов поиска CIND CINDERELLA и PLI-CIND;
2. Разработать и реализовать алгоритм валидации CIND, позволяющий проверять выполнимость заданной условной зависимости включения на конкретных данных, интегрировать его в Desbordante с тестами и примерами использования;
3. Реализовать альтернативный алгоритм поиска CIND, интегрировать его в Desbordante с тестами и примерами использования;
4. Провести экспериментальное исследование: подтвердить, что оптимизации сохраняют выдачу алгоритмов без изменений, оценить влияние оптимизаций на время работы и потребление памяти, измерить производительность валидатора CIND и сравнить реализованные алгоритмы поиска.

2. Обзор

Данная работа продолжает исследование, начатое в бакалаврской работе Абросимова [31], в которой была выполнена первоначальная реализация алгоритмов CINDERELLA и PLI-CIND. Для того чтобы данный текст был самодостаточным, ниже приводятся необходимые определения и описания алгоритмов. Если явно не указан другой источник, все определения и обозначения приводятся по [5].

2.1. Условные зависимости включения

2.1.1. Основные обозначения

Пусть R_1, R_2 — реляционные схемы над фиксированными наборами атрибутов $A_1, A_2, \dots, A_k$. Множество допустимых значений каждого атрибута A обозначим как $\text{dom}(A)$. Пусть I_1, I_2 — экземпляры схем R_1, R_2 соответственно. Каждый экземпляр I состоит из набора кортежей t таких, что $t[A] \in \text{dom}(A)$ для каждого атрибута $A \in R$. Пусть X, X_P — множества атрибутов R_1 , а Y, Y_P — множества атрибутов R_2 . Обозначим за $t[X]$ проекцию кортежа t на атрибуты X .

Замечание. На практике I_1, I_2 представляют собой таблицы в реляционной базе данных, а t — строку таблицы.

2.1.2. Зависимость включения

Определение 1 (Зависимость включения). *Зависимостью включения (Inclusion Dependency, IND) называется отношение $R_1[X] \subseteq R_2[Y]$, которое выполняется на всей таблице. То есть, $\forall t_1 \in I_1 \exists t_2 \in I_2 : t_1[X] = t_2[Y]$.*

Содержательно IND означает, что множество значений атрибутов X в первой таблице является подмножеством значений атрибутов Y во второй. Кортеж $t_1 \in I_1$ удовлетворяет IND, если существует ссылочный кортеж $t_2 \in I_2$, для которого $t_1[X] = t_2[Y]$. Атрибуты X и Y называются *атрибутами включения*.

На рис. 1 представлен пример зависимости включения. Каждое значение столбца PK таблицы $Table1$ присутствует в столбце PK таблицы $Table2$, что формально записывается как $IND\ Table1[PK] \subseteq Table2[PK]$. Стрелки на рисунке показывают, какому кортежу из $Table2$ соответствует каждый кортеж из $Table1$.

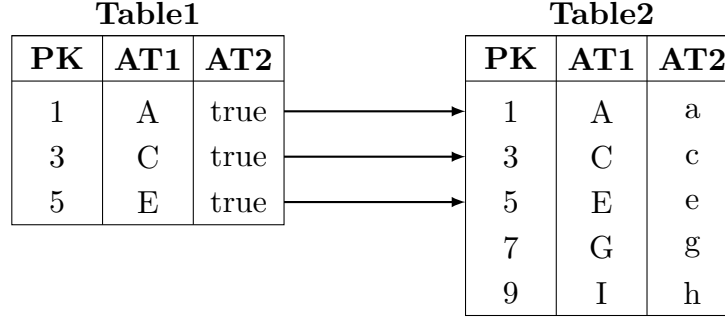


Рис. 1: Пример IND: $Table1.PK \subseteq Table2.PK$

Зависимости включения применяются при обнаружении внешних ключей [25] и поддержании ссылочной целостности [24]. Задача автоматического поиска всех выполняющихся IND является NP-трудной [19].

2.1.3. Приближённая зависимость включения

На практике зависимости включения не всегда выполняются на всех данных. Для описания таких случаев вводится ослабленное понятие приближённой зависимости включения.

Определение 2 (Приближённая зависимость включения). *Приближённой зависимостью включения (Approximate INclusion Dependency, AIND) называется такая IND, которая выполняется на непустом подмножестве кортежей из I_1 . То есть, $\exists t_1 \in I_1, t_2 \in I_2 : t_1[X] = t_2[Y]$. Обозначается как $R_1[X] \subseteq' R_2[Y]$.*

Для приближённой зависимости включения определена метрика ошибки — доля кортежей, на которых зависимость не выполняется.

Определение 3 (Ошибка AIND). *Ошибкой AIND называют число $\varepsilon \in [0, 1]$, показывающее долю кортежей, на которых соответствующая*

щая IND не выполняется. Вычисляется по формуле:

$$\varepsilon = 1 - \frac{|I_1[R_1[X] \subseteq' R_2[Y]]|}{|I_1|},$$

где $I_1[R_1[X] \subseteq' R_2[Y]]$ — множество всех кортежей из I_1 , для которых зависимость $R_1[X] \subseteq R_2[Y]$ выполняется.

На рис. 2 представлен пример приближённой зависимости включения. Кортежи со значениями $PK \in \{1, 3, 5\}$ удовлетворяют IND , а кортежи со значениями $PK \in \{2, 4\}$ — нет, поскольку значения 2 и 4 отсутствуют в столбце PK таблицы *Table2*. На рисунке невключённые строки выделены цветом. Таким образом, $AIND\ Table1[PK] \subseteq' Table2[PK]$ выполняется с ошибкой $\varepsilon = 1 - 3/5 = 0,4$.

Table1				Table2		
PK	AT1	AT2		PK	AT1	AT2
1	A	true	→	1	A	a
2	B	false		3	C	c
3	C	true		5	E	e
4	D	false		7	G	g
5	E	true		9	I	h

Рис. 2: Пример AIND: $Table1.PK \subseteq Table2.PK$ с метрикой ошибки $\varepsilon = 0,4$

Приближённые зависимости дают агрегированную оценку доли нарушений, однако не раскрывают того, на каком именно подмножестве данных зависимость выполняется. Для преодоления этого ограничения вводится понятие условной зависимости включения.

2.1.4. Таблица условий

Прежде чем определить условную зависимость включения, необходимо ввести понятие таблицы условий — центрального компонента, определяющего подмножество кортежей, на которых зависимость должна выполняться.

Определение 4 (Таблица условий). *Таблица условий (Pattern Table)* T_P состоит из кортежей над атрибутами X_P и Y_P . Для каждого

такого кортежа t_P выполняется:

$$\forall A \in X_P \cup Y_P : t_P[A] = a \in \text{dom}(A) \vee t_P[A] = \text{—},$$

где ‘—’ является специальным значением, обозначающим отсутствие ограничения на данный атрибут. Такие кортежи называются шаблонами, или условиями.

Говорят, что кортеж $t_1 \in I_1$ удовлетворяет условию t_P , если $\forall A \in X_P : t_1[A] = t_P[A] \vee t_P[A] = \text{—}$. Атрибуты X_P, Y_P называются *атрибутами условия*. Важно, что множества X и X_P , а также Y и Y_P , не пересекаются.

В качестве примера рассмотрим шаблон $t_P = (AT1 = \text{—}, AT2 = \text{true})$ при $X_P = \{AT1, AT2\}$. Этот шаблон означает: «атрибут $AT1$ может принимать любое значение, а $AT2$ должен быть равен true». Для данных из рис. 2 условию удовлетворяют кортежи с $PK \in \{1, 3, 5\}$, и все они являются включёнными.

2.1.5. Условная зависимость включения

В литературе используются два варианта определения условной зависимости включения. Исходное определение было предложено Браво и Фанем [5]; на нём основаны алгоритмы CINDERELLA и PLI-CIND. Алгоритм Cure [11] опирается на расширенный вариант, допускающий дизъюнкции значений в таблице условий и условия на атрибуты обеих таблиц. Ниже приводятся оба определения.

Определение 5 (Условная зависимость включения по [5]). *Условной зависимостью включения (Conditional INclusion Dependency, CIND) называется отношение вида:*

$$\phi : (R_1[X; X_P] \subseteq R_2[Y; Y_P], T_P).$$

Оно состоит из $IND R_1[X] \subseteq R_2[Y]$, называемой *встроенной IND*, и таблицы условий T_P , которая определяет подмножество кортежей, на которых эта IND должна выполняться.

CIND ϕ выполняется для экземпляров I_1, I_2 , если выполнены следу-

ющие условия:

Условие выбора для I_1 . Пусть $t_1 \in I_1$ удовлетворяет какому-либо условию $t_P \in T_P$. Тогда t_1 должна удовлетворять встроенной IND.

Условие требования для I_2 . Пусть $t_1 \in I_1$ удовлетворяет условию $t_P \in T_P$ и встроенной IND, то есть $\exists t_2 \in I_2 : t_1[X] = t_2[Y]$. Тогда t_2 также должна удовлетворять t_P .

Заметим, что условие выбора для I_1 включает в себя условие требования для I_2 . Более того, шаблон t_P , для которого $\forall A \in Y_P : t_P[A] = \text{‘—’}$, является полностью валидным, то есть условие требования для I_2 выполняется тривиально. Поэтому в дальнейшем при поиске CIND будем учитывать только условие выбора для I_1 .

Алгоритм Cure [11] использует расширенный вариант определения CIND, допускающий дизъюнкцию значений в ячейках таблицы условий.

Определение 6 (Расширенная условная зависимость включения по [11]). *Расширенной условной зависимостью включения называется CIND $\phi : (R_1[X; X_P] \subseteq R_2[Y; Y_P], T_P)$, отличающаяся от определения 5 тем, что в качестве значений шаблонов в таблице условий T_P допускаются дизъюнкции: для каждого шаблона t_P и атрибута $A \in X_P \cup Y_P$ значение имеет вид*

$$t_P[A] = \{a_1, \dots, a_k\} \subseteq \text{dom}(A) \vee t_P[A] = \text{‘—’}.$$

Кортеж t удовлетворяет такому шаблону, если для каждого атрибута $A \in X_P$ выполняется $t[A] \in t_P[A]$ либо $t_P[A] = \text{‘—’}$.

Определение 5 является частным случаем определения 6, в котором каждая дизъюнкция содержит ровно одно значение. Использование дизъюнкций позволяет получать более компактные таблицы условий: вместо нескольких шаблонов, различающихся лишь значениями одного атрибута, записывается один шаблон с дизъюнкцией этих значений. Кроме того, расширенное определение задействует условия на атрибуты обеих таблиц X_P и Y_P , тогда как алгоритмы CINDERELLA и PLI-CIND, опирающиеся на исходное определение, на практике ищут условия только на атрибуты X_P левой таблицы (см. замечание о три-

виальности условия требования для I_2 выше).

На рис. 3 представлен пример CIND. Рассмотрим встроенную IND $Table1[PK] \subseteq Table2[PK]$ и таблицу условий $T_P = \{(AT1 = \text{---}, AT2 = \text{true})\}$. Условию $AT2 = \text{true}$ удовлетворяют кортежи с $PK \in \{1, 3, 5\}$. Для каждого из них в $Table2$ существует кортеж с совпадающим значением PK , следовательно, условие выбора выполняется и данная CIND валидна. При этом невключённые кортежи ($PK \in \{2, 4\}$) не удовлетворяют условию $AT2 = \text{true}$ и потому не нарушают CIND. На рисунке строки, удовлетворяющие условию, выделены цветом.

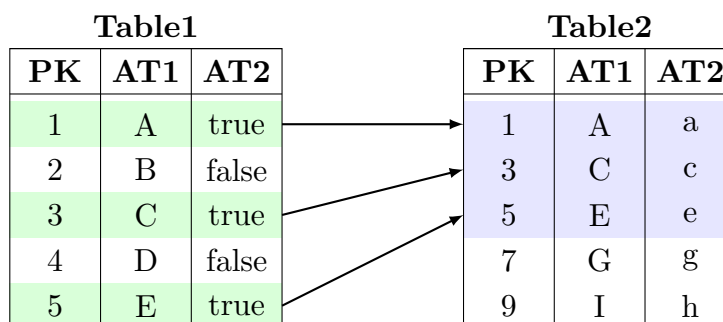


Рис. 3: Пример CIND: $Table1.PK \subseteq Table2.PK$ с таблицей условий $T_P = \{(AT1 = \text{---}, AT2 = \text{true})\}$

2.1.6. Метрики качества условий

Полностью валидных и полностью полных условий на практике обычно не существует, поэтому алгоритмы поиска CIND фильтруют кандидатов по тем или иным численным характеристикам качества. Конкретный набор характеристик определяется алгоритмом. CINDERELLA и PLI-CIND опираются на две взаимодополняющие метрики — валидность и полноту, аналогичные точности и полноте классификатора в машинном обучении. Алгоритм Cure использует более простой критерий — порог минимальной поддержки *support*, выражающий абсолютное число кортежей, подтверждающих шаблон; этот параметр подробно рассматривается при описании самого алгоритма (раздел 2.3.3). Ниже даются формальные определения валидности и полноты — наиболее общих из перечисленных характеристик, применимых к любому условию.

Прежде чем определить метрики, введём необходимые обозначения.

Пусть дана CIND ϕ и экземпляры I_1, I_2 . Обозначим за I_ϕ множество кортежей из I_1 , удовлетворяющих встроенной IND. Для ненормализованных таблиц существенную роль играют *группы включённых кортежей*. Пусть $g_x = \{t \mid t \in I_1 \wedge t[X] = x\}$ — группа кортежей из I_1 , у которых значение на атрибутах включения X равно x . Группа называется *включённой*, если хотя бы один кортеж из неё удовлетворяет IND. Обозначим G_1 — множество всех групп в I_1 , G_ϕ — множество включённых групп. Наконец, $I[t_P]$ — множество кортежей из I , удовлетворяющих условию t_P , а $G[t_P]$ — множество таких групп.

Определение 7 (Валидное условие). *Условие называется валидным, если все кортежи (группы), которые ему удовлетворяют, являются включёнными. Является аналогом точности (precision) классификатора из теории машинного обучения.*

Определение 8 (Полное условие). *Условие называется полным, если ему удовлетворяют все включённые кортежи (группы). Для групп такое свойство может также называться покрытием (covering). Является аналогом полноты (recall) классификатора.*

В реальности получить полностью валидные или полные условия практически невозможно. Поэтому определим метрики валидности и полноты как метрики, которые возможно посчитать.

Определение 9 (Валидность условия). *Валидность условия $\text{valid}(t_P) \in [0, 1]$ вычисляется по формуле*

$$\text{valid}(t_P) = \frac{|I_\phi[t_P]|}{|I_1[t_P]|}$$

для строковых условий, и по формуле

$$\text{valid}_g(t_P) = \frac{|G_\phi[t_P]|}{|G_1[t_P]|}$$

для групповых. Условие t_P называется γ -валидным, если его валидность больше γ .

Определение 10 (Полнота условия). *Полнота условия completeness*

$(t_P) \in [0, 1]$ вычисляется по формуле

$$\text{completeness}(t_P) = \frac{|I_\phi[t_P]|}{|I_\phi|}$$

для строковых условий, и по формуле

$$\text{completeness}_g(t_P) = \frac{|G_\phi[t_P]|}{|G_\phi|}$$

для групповых. Условие t_P называется δ -полным, если его полнота больше δ .

Проиллюстрируем метрики на примере. Вернёмся к данным из рис. 2. Условие $t_P = (AT2 = \text{true})$ удовлетворяется тремя кортежами из I_1 : $\{1, 3, 5\}$. Все три являются включёнными, поэтому $\text{valid}(t_P) = 3/3 = 1,0$. Включённых кортежей всего три, и все они удовлетворяют условию, поэтому $\text{completeness}(t_P) = 3/3 = 1,0$. Данное условие является одновременно полностью валидным и полностью полным.

Теперь рассмотрим условие $t'_P = (AT1 = 'A')$. Ему удовлетворяет один кортеж с $PK = 1$, который является включённым: $\text{valid}(t'_P) = 1/1 = 1,0$. Однако $\text{completeness}(t'_P) = 1/3 \approx 0,33$ — условие покрывает лишь треть включённых кортежей.

Используя эти метрики, можно задавать ограничения на найденные условия и фильтровать как те, которые охватывают слишком много невалидных кортежей, так и те, которые применяются на чрезмерно малой выборке данных.

*Замечание: в дальнейшем условия, метрики которых посчитаны для групп, будут называться **групповыми** условиями. В свою очередь те условия, метрики которых посчитаны для обычного представления таблицы, будут называться **строковыми** условиями.*

Таким образом, CIND представляет собой составной примитив, состоящий из AIND и таблицы условий. При поиске CIND необходимо сначала обнаружить AIND, а затем для каждой из них отдельно искать подходящие условия. Поскольку алгоритмы поиска AIND уже реализованы в Desbordante, дальнейшее изложение сосредоточено на ал-

горитмах поиска условий. Метрики качества условия могут вычисляться как для отдельных кортежей (строковые условия), так и для групп кортежей с одинаковыми значениями атрибутов включения (групповые условия).

2.2. Алгоритмы поиска AIND

Как было отмечено выше, поиск CIND начинается с обнаружения приближённых зависимостей включения. Нахождение всего множества зависимостей включения для заданного набора данных — одна из наиболее вычислительно сложных задач профилирования данных [23]. Для её решения разработан ряд специализированных алгоритмов, среди которых наиболее известны следующие:

- **SPIDER** [21] — алгоритм, основанный на сортировке значений атрибутов и последующем слиянии отсортированных списков. Он не требует загрузки всех данных в оперативную память: отсортированные списки могут храниться на диске и считываться последовательно, что делает его применимым к наборам данных, существенно превышающим объём доступной памяти. Алгоритм эффективно обнаруживает как точные, так и приближённые зависимости включения;
- **BINDER** [20] — алгоритм, использующий стратегию «разделяй и властвуй» для сокращения пространства поиска. Применяет фильтры Блума для предварительного отсека заведомо невыполняющихся кандидатов, что позволяет эффективно работать с большими наборами данных;
- **MIND** [15] — алгоритм поиска унарных зависимостей включения, основанный на инвертированных индексах. Отличается простотой реализации, однако при большом количестве уникальных значений размер индексов существенно возрастает;

- **FAIDA** [22] — алгоритм, использующий хэш-функции для приближённого сравнения множеств значений атрибутов. FAIDA гарантированно находит все выполняющиеся IND, но может выдавать ложноположительные результаты. Ориентирован на поиск точных IND и не поддерживает приближённые зависимости.

В Desbordante на момент начала данной работы реализованы SPIDER, FAIDA и MIND. Поскольку для пайплайна CIND необходим алгоритм, способный обнаруживать приближённые зависимости включения, FAIDA не подходит — он поддерживает только точные IND. Алгоритм MIND, хотя и позволяет искать AIND, был реализован в Desbordante сравнительно недавно, и его эффективность в рамках данной реализации не была экспериментально подтверждена. По этим причинам в качестве алгоритма поиска AIND для пайплайна CIND был выбран SPIDER — он хорошо зарекомендовал себя как для точных, так и для приближённых зависимостей включения, обладает предсказуемым потреблением памяти и уже имеет проверенную реализацию в Desbordante.

2.3. Алгоритмы поиска условий CIND

Алгоритмы поиска условий для CIND были впервые представлены в статье Баукманн и др. [18]. Авторы предложили два алгоритма — CINDERELLA и PLI-CIND, основанных на различных принципах, но объединённых общей двухфазной архитектурой. Альтернативный подход — алгоритм Cure — был независимо предложен О. Кюре в работе [11]; в данной работе рассматриваются все три алгоритма.

Все три рассматриваемых алгоритма принимают на вход результат работы алгоритма поиска AIND — список приближённых зависимостей включения, и для каждой найденной AIND отдельно ищут таблицу условий T_P . CINDERELLA и PLI-CIND формируют таблицу из γ -валидных и δ -полных условий, а ключевое различие между ними заключается в стратегии поиска кандидатов в условия — корректных

шаблонов, которые потенциально могут войти в таблицу условий, но метрики качества которых ещё не вычислены.

Оба алгоритма опираются на идею *внешнего левого соединения* (left outer join) двух таблиц по атрибутам включения. Результат такого соединения позволяет для каждого кортежа из I_1 определить, является ли он включённым, то есть имеет ли соответствующий кортеж в I_2 . С целью упрощения дальнейшего изложения каждый алгоритм сначала описывается в версии для поиска групповых условий, а затем указываются модификации для строковых условий.

2.3.1. Алгоритм CINDERELLA

Алгоритм Conditional INclusion DEpendency REcognition Leveraging deLimited Apriori (CINDERELLA) основан на поиске ассоциативных правил [18]. Неформально ассоциативные правила имеют такой смысл: «кто покупает X , тот зачастую покупает и Z ». В контексте CINDERELLA эти правила приобретают следующую интерпретацию: «строки таблицы, чьи значения атрибутов условий равны x , как правило, являются включёнными».

Для поиска ассоциативных правил используется алгоритм Apriori [3], который представляет таблицу в виде *контейнеров* (baskets). В каждый контейнер помещаются все пары вида (атрибут условия, значение), соответствующие одной группе (или строке — для строковых условий), а также индикатор включения, если соответствующая группа является включённой.

Apriori использует метрики support и confidence для сужения области поиска и вывода нужных правил. В контексте CIND этими метриками выступают полнота и валидность. Область поиска можно сократить по метрике полноты, пользуясь следующим инвариантом: с ростом количества атрибутов в условии его полнота не возрастает. Таким образом, алгоритм получает все наборы кандидатов с необходимой полнотой, после чего фильтрует их по валидности, формируя итоговый список условий.

Алгоритм CINDERELLA основан на одной из вариаций Apriori — алгоритме MultipleJoins [18]. Его ключевая оптимизация состоит в том, что нас интересуют только правила, у которых атрибут включения имеет положительное значение, то есть нас интересуют только условия для включённых групп. Благодаря этому CINDERELLA при поиске кандидатов может использовать одно соединение вместо трёх, как в оригинальном MultipleJoins, за счёт чего сильно сужается область поиска кандидатов и повышается производительность.

Опишем работу алгоритма более детально. Пусть L_k — множество наборов элементов длины k , обладающих достаточной полнотой ($\geq \delta$), а C_k — множество кандидатов в L_k .

Для начала за один проход по всем контейнерам вычисляется L_1 — множество отдельных элементов с полнотой не ниже δ . Затем на каждой последующей итерации формируется C_k из L_{k-1} : кандидат формируется из двух наборов, у которых первые $k-2$ элементов одинаковы, а последний элемент первого набора меньше последнего элемента второго. Таким образом поддерживается отсортированный порядок элементов в наборе. Далее необходимо определить, является ли получившийся набор элементов допустимым: для этого проверяется, что все подмножества длины $k-1$, содержащие индикатор включения, лежат в L_{k-1} . У получившихся кандидатов заново вычисляется полнота, и по ней фильтруются наборы, формируя L_k . Процесс повторяется до тех пор, пока L_k не окажется пустым.

Получив все наборы элементов с необходимой полнотой, остаётся лишь выбрать из них те, что имеют достаточную валидность. Это можно делать в процессе вычисления полноты каждого набора элементов. Чтобы применять такой алгоритм для поиска строковых условий, достаточно формировать по контейнеру на каждую отдельную строку таблицы, а не на группу.

CINDERELLA осуществляет поиск кандидатов «в ширину» (breadth-first), последовательно наращивая длину условий. Это позволяет эффективно отсекалть бесперспективные ветви поиска благодаря анитонности полноты. Вместе с тем для проверки допустимости набора

кандидатов на каждом уровне необходимо хранить хэш-таблицы всех предыдущих уровней, что обеспечивает высокую скорость работы ценой значительного потребления оперативной памяти.

2.3.2. Алгоритм PLI-CIND

Второй алгоритм — PLI-based Algorithm (PLI-CIND) — основан на идее пересечения позиционных списков атрибутов (Position Lists Intersection, PLI) [18]. Позиционный список атрибута — это преобразованное представление столбца таблицы, в котором каждому значению атрибута A в строке t сопоставляется набор индексов строк (или групп), содержащих это значение. Позиционные списки можно объединять, пересекая их между собой, формируя тем самым позиционные списки для комбинаций значений нескольких атрибутов. Данное представление широко используется в алгоритмах поиска функциональных зависимостей, например TANE [30].

PLI-CIND, в отличие от CINDERELLA, не пытается на каждой итерации искать все условия одинаковой длины. Напротив, он использует поиск «в глубину» (depth-first): пытается расширить текущее условие до тех пор, пока это возможно. Алгоритм разбит на две фазы.

Фаза 1. Построение позиционных списков. Первая фаза выполняется за один проход по таблице: для каждого значения каждого атрибута условия строится позиционный список, содержащий только индексы включённых групп (или строк). Каждое значение атрибута проверяется на полноту — если бы это значение сформировало условие, имело бы оно достаточную полноту? В итоге остаются только те списки, которые имеют нужную полноту.

Фаза 2. Рекурсивный поиск условий. Рекурсия запускается из каждого атрибута отдельно. Пока текущий позиционный список не пуст, алгоритм пытается расширить его добавлением нового атрибута. Расширение выполняется пересечением текущего списка со списком позиций этого атрибута. В процессе пересчитываются полнота и валидность: полные списки отправляются по рекурсии дальше, а валидные —

заносятся в ответ.

Реализации для различных типов условий различаются лишь типом индексов, которые используются на первой фазе при формировании позиционных списков: для строковых условий индексами служат номера строк, для групповых — номера групп.

2.3.3. Алгоритм Cure

Алгоритм Cure был предложен О. Кюре [11] в контексте повышения качества фармацевтических баз данных и реализует принципиально иную стратегию поиска условий по сравнению с CINDERELLA и PLI-CIND. Вместо обхода решётки кандидатов и фильтрации по валидности и полноте Cure управляется единственным параметром — порогом поддержки *support*, обозначающим минимальное число кортежей, подтверждающих шаблон. Кроме того, в отличие от CINDERELLA и PLI-CIND, Cure допускает дизъюнкцию значений для атрибутов из правой части таблицы условий, что позволяет получать более компактные итоговые таблицы (см. пример далее).

Алгоритм принимает на вход список AIND и базу данных, состоит из двух фаз — `discoverCIND` и `minimalCover` — и в [11] сформулирован в терминах SQL-запросов к реляционной базе данных.

Фаза 1. `discoverCIND`. Для каждой AIND $R_1[X] \subseteq R_2[Y]$ алгоритм перебирает все пары (A_p, B_p) атрибутов отношений R_1 и R_2 , исключая попарные соответствия атрибутов включения — пары вида (X_i, Y_i) . Для каждой оставшейся пары формируется и выполняется SQL-запрос вида:

```
select distinct r1.A_p, r2.B_p
  from R1 r1, R2 r2
 where r1.X_1 = r2.Y_1 and ... and r1.X_n = r2.Y_n
 group by r1.A_p, r2.B_p
having count(*) >= support;
```

Запрос соединяет R_1 и R_2 по атрибутам включения и возвращает все пары значений (a, b) атрибутов A_p, B_p , для которых не менее s кортежей R_1 со значением a имеют соответствие в R_2 со значением b . Каж-

дая такая пара образует отдельный шаблон в накопительной структуре `cindList`.

Фаза 2. `minimalCover`. По завершении первой фазы `cindList` обычно содержит избыточные шаблоны: разные значения B_p , соответствующие одной и той же паре (A_p, a) , хранятся как отдельные строки. Функция `minimalCover` строит минимальное покрытие `cindList'`: при добавлении нового шаблона алгоритм ищет в `cindList'` запись с совпадающей левой частью и в случае совпадения объединяет значения правой части с существующей записью — либо как новые значения других атрибутов, либо как дизъюнкция значений одного и того же атрибута. Шаблоны без совпадений добавляются в `cindList'` как новые записи.

Проиллюстрируем работу алгоритма на примере из [11]. Рассмотрим AIND $ind_5 : R[r_1, r_2] \subseteq S[s_1, s_2]$, порог поддержки $support = 2$ и базу данных, представленную на рис. 4.

R				S				
r_1	r_2	r_3	r_4	s_1	s_2	s_3	s_4	s_5
1	1	a	x_1	1	1	b	c	y_1
1	2	a	x_2	1	2	b	c	y_2
1	3	a	x_3	1	3	b	c	y_3
2	1	x_4	d	2	1	y_4	e	y_6
2	2	x_5	d	2	2	y_7	e	y_8
2	3	x_6	d	2	3	y_9	f	y_{10}
2	4	x_7	d	2	4	y_{11}	e	y_{12}

Рис. 4: Входные данные для примера работы Cuge: отношения R и S

Для пары атрибутов (r_3, s_3) алгоритм сгенерирует следующий запрос:

```
select distinct R.r3, S.s3 from R, S
where S.s1 = R.r1 and S.s2 = R.r2
group by S.s3, R.r3 having count(*) >= 2;
```

Ответом запроса является единственная пара (a, b) : три кортежа R со значением a на r_3 соединяются с тремя кортежами S со значением b на s_3 . После обработки всех пар атрибутов первая фаза формирует таблицу из четырёх шаблонов, представленную на рис. 5.

r_1	r_2	r_3	r_4	s_1	s_2	s_3	s_4	s_5
		a				{b}		
		a					{c}	
			d				{e}	
			d				{f}	

Рис. 5: Таблица условий после фазы discoverCIND

После применения `minimalCover` избыточные шаблоны сворачиваются. Шаблоны с одинаковым значением $r_3 = a$ объединяются в одну строку. Для $r_4 = d$ оба шаблона указывают на один и тот же атрибут s_4 , и их значения сливаются в дизъюнкцию $\{e, f\}$. Итоговая таблица условий приведена на рис. 6.

r_1	r_2	r_3	r_4	s_1	s_2	s_3	s_4
		a				{b}	{c}
			d				{e, f}

Рис. 6: Таблица условий после фазы minimalCover

Таким образом, итоговая CIND имеет вид $\phi_5 : (R[r_1, r_2; r_3, r_4] \subseteq S[s_1, s_2; s_3, s_4], T_P)$ с приведённой выше таблицей условий T_P .

2.3.4. Сравнение алгоритмов

Среди трёх рассмотренных алгоритмов нет универсально лучшего: каждый имеет свои преимущества и ограничения.

CINDERELLA ищет кандидатов «в ширину», что позволяет существенно снизить область поиска условий. Вместе с тем этот алгоритм должен хранить намного больший объём данных в процессе работы: для проверки правильности набора необходимо хранить кандидатов всех размеров в виде хэш-таблиц, что обеспечивает высокую скорость работы ценой существенного потребления памяти.

PLI-CIND, напротив, потребляет память только на поддержание текущего кандидата и его позиционного списка, что делает совокупное потребление памяти не зависящим от числа кандидатов. Однако поиск

«в глубину» никак не ограничивает область поиска для расширения кандидата, за счёт чего алгоритм не так эффективен по времени.

Cure, в отличие от первых двух алгоритмов, не обходит решётку кандидатов: он перебирает пары атрибутов условия и для каждой пары вычисляет совместно встречающиеся значения через соединение таблиц. Алгоритм выдаёт компактные шаблоны с дизъюнкциями в правой части и управляется единственным параметром — порогом поддержки.

Различия в выдаче алгоритмов проявляются на конкретных данных. Рассмотрим тот же набор данных, что использовался для иллюстрации работы Cure (рис. 4), и запустим на нём CINDERELLA или PLI-CIND с типичными порогами $\gamma = 0,9$ и $\delta = 0,05$ для AIND $R[r_1, r_2] \subseteq S[s_1, s_2]$.

Группировка R по значениям атрибутов включения (r_1, r_2) даёт семь групп, и все они являются включёнными. Кандидат $r_3 = a$ покрывает три включённые группы из семи, кандидат $r_4 = d$ — четыре; оба имеют валидность 1,0 и проходят пороги γ и δ . Никакая комбинация этих кандидатов не даёт непустого множества групп, поэтому итоговая таблица условий содержит ровно две строки и приведена на рис. 7.

r_1	r_2	r_3	r_4	s_1	s_2	s_3	s_4
		a	d				

Рис. 7: Таблица условий, полученная CINDERELLA/PLI-CIND на данных из рис. 4

Сравнивая эту таблицу с выдачей Cure (рис. 6) на тех же данных, можно выделить два ключевых различия. Во-первых, CINDERELLA и PLI-CIND заполняют только атрибуты левой таблицы X_P , а на месте атрибутов правой таблицы Y_P ставится ‘—’. Cure же выдаёт значения на обеих сторонах, поскольку каждая пара (a, b) возникает естественным образом при соединении R и S . Во-вторых, два шаблона $r_4 = d, s_4 = e$ и $r_4 = d, s_4 = f$, которые Cure сворачивает в одну строку с дизъюнкцией $s_4 = \{e, f\}$, у CINDERELLA и PLI-CIND в принципе не возникают: обход решётки кандидатов идёт только по атрибутам X_P , а механизм объединения значений в дизъюнкцию отсутствует.

Таким образом, выбор алгоритма определяется характеристиками задачи: CINDERELLA подходит при необходимости минимизации времени работы и достаточном запасе памяти, PLI-CIND — при ограниченных ресурсах оперативной памяти, а Cure — когда требуется компактное представление таблицы условий с возможностью дизъюнкций и управление единственным простым параметром.

2.4. Архитектура Desbordante

Desbordante [16] — высокопроизводительный инструмент профилирования данных с открытым исходным кодом, реализованный на языке C++. Проект разрабатывается группой исследователей из Санкт-Петербургского государственного университета с 2019 года. По своему назначению Desbordante во многом схож с Metanome [13], однако обладает рядом отличительных особенностей: реализация на C++ позволяет использовать низкоуровневые оптимизации; инструмент доступен в качестве pip-пакета для языка Python и предоставляет интерфейс командной строки; несмотря на меньший возраст проекта, в нём реализовано больше примитивов профилирования, чем в Metanome.

Архитектура Desbordante организована в виде многоуровневой системы, общая схема которой представлена на рис. 8. На нижнем уровне расположен слой ввода данных, поддерживающий чтение из CSV-файлов и pandas DataFrame. Данные преобразуются в единое внутреннее представление, включающее реляционную схему, систему типов и индексные структуры — позиционные списки (PLI) и индексы столбцов. Над слоем данных располагается ядро системы — библиотека алгоритмов, организованная вокруг базового класса `Algorithm`, предоставляющего единый жизненный цикл: загрузка данных, конфигурация параметров, выполнение и получение результатов. Каждый алгоритм реализует поиск или валидацию определённого типа зависимостей. На верхнем уровне находятся пользовательские интерфейсы: Python-привязки на основе `rybind11` и интерфейс командной строки.

На момент начала данной работы в Desbordante реализована под-

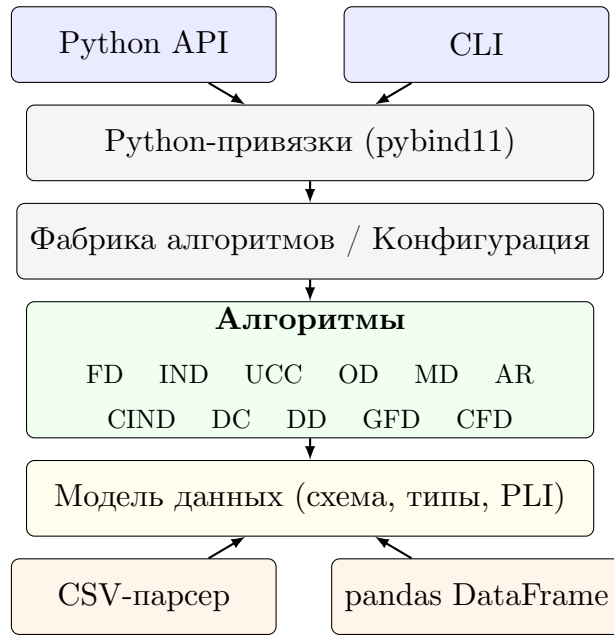


Рис. 8: Архитектура инструмента Desbordante

держка более 30 алгоритмов поиска и валидации зависимостей данных, включая функциональные зависимости (FD), зависимости включения (IND), уникальные комбинации столбцов (UCC), порядковые зависимости (OD), зависимости соответствия (MD), ассоциативные правила (AR) и ряд других примитивов. Поддержка условных зависимостей включения (CIND) была добавлена в рамках бакалаврской работы [31], однако реализованные алгоритмы не были оптимизированы по производительности, задача валидации CIND не рассматривалась, а альтернативные алгоритмы поиска условий не были реализованы. Настоящая работа посвящена оптимизации этих алгоритмов, реализации валидатора CIND и реализации альтернативного алгоритма поиска условий — Cure.

3. Описание решения

3.1. Иерархия классов

Общая иерархия классов, связанных с условными зависимостями включения, представлена на рис. 9. Базовый класс `Algorithm` определяет единый жизненный цикл всех алгоритмов в `Desbordante`: загрузка данных (`LoadData`), конфигурация параметров (`SetOption`) и выполнение (`Execute`). Каждый алгоритм реализует три чисто виртуальных метода: `LoadDataInternal`, `ExecuteInternal` и `ResetState`.

Класс `CindAlgorithm` реализует полный пайплайн поиска CIND. Внутри он создаёт экземпляр алгоритма `Spider` для обнаружения приближённых зависимостей включения, а затем передаёт найденные AIND одному из майнеров условий — `Cinderella`, `PliCind` или `CureCind`, выбираемому через параметр `AlgoType`. Все три майнера наследуются от абстрактного класса `CindMiner`, который определяет интерфейс для поиска условий.

Класс `CINDVerifier`, реализованный в рамках данной работы, также наследуется от `Algorithm` и предоставляет функциональность валидации — проверки того, выполняется ли заданная CIND на конкретных данных. В отличие от `CindAlgorithm`, валидатор не выполняет поиск, а проверяет одну конкретную зависимость с заданным условием.

Обозначения цветов на диаграмме:

- **Серый** — классы, не затронутые в данной работе (`Algorithm`, `CindAlgorithm`, `CindMiner`);
- **Жёлтый** — классы, в которых проведены оптимизации (`Cinderella`, `PliCind`, `ItemsetNode`, `PositionListsSet`);
- **Зелёный** — классы, реализованные в данной работе (`CINDVerifier`, `CureCind`).

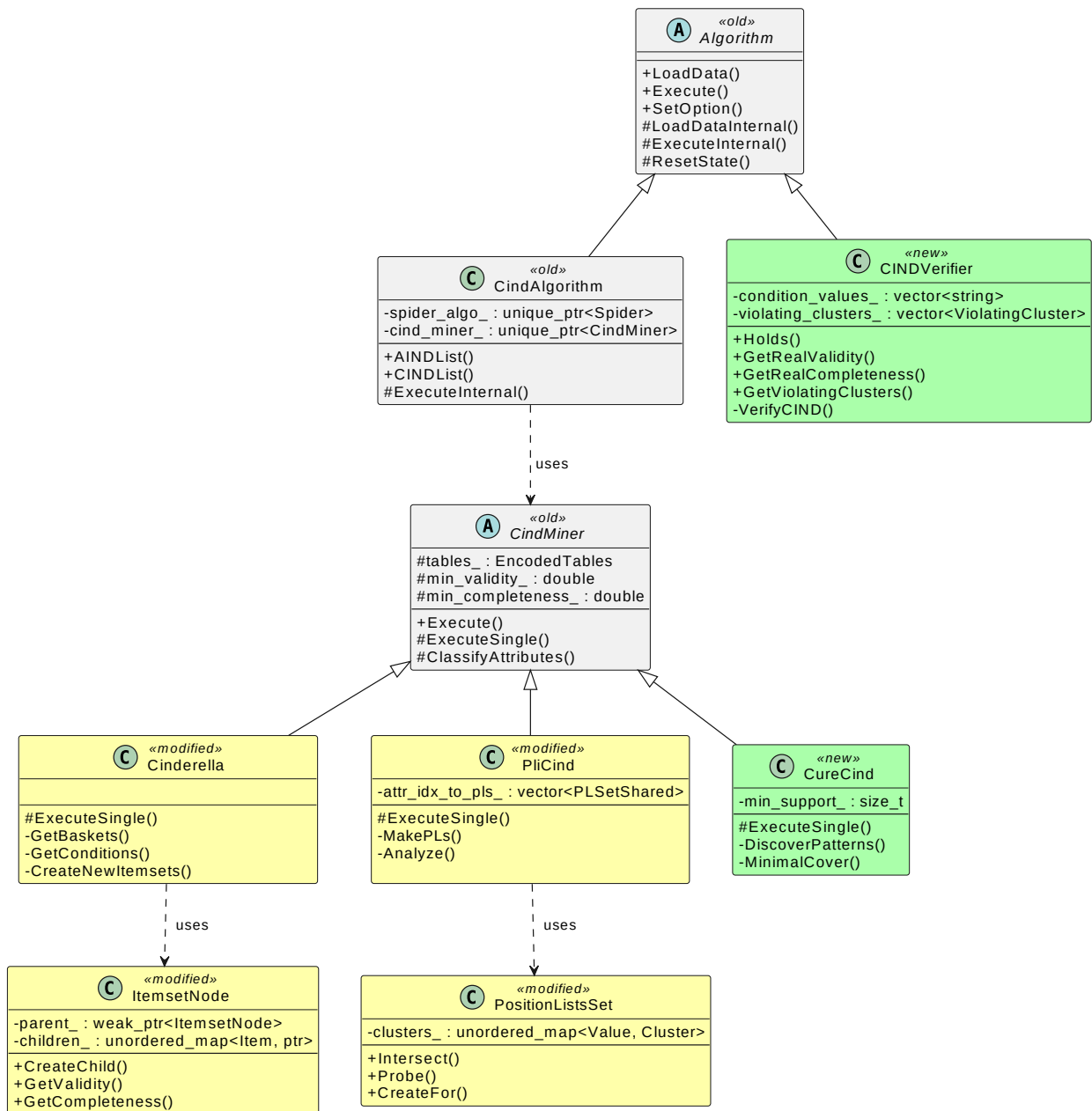


Рис. 9: UML-диаграмма классов поиска и валидации CIND.

3.2. Реализация алгоритма Cure

Алгоритм Cure реализован в виде класса `CureCind`, наследующего от абстрактного майнера условий `CindMiner` и переопределяющего его виртуальный метод `ExecuteSingle`.

В оригинальной формулировке [11] алгоритм опирается на SQL-запросы к реляционной базе данных. В Desbordante данные хранятся

в оперативной памяти в виде закодированных таблиц (`EncodedTables`), поэтому соединение и подсчёт пар значений реализованы напрямую через хэш-таблицы.

Алгоритм работает в две фазы.

Фаза 1 (`DiscoverPatterns`). Для каждой приближённой зависимости включения $R_1[X] \subseteq R_2[Y]$ строится индекс `rhs_index` — хэш-таблица, ключом которой служат значения атрибутов Y во второй таблице, а значением — список индексов соответствующих строк. Затем для каждой пары атрибутов условия (A_p, B_p) , где A_p — атрибут R_1 вне X , а B_p — атрибут R_2 вне Y , выполняется проход по строкам R_1 : для каждой строки через `rhs_index` извлекаются строки R_2 с тем же ключом включения, и для полученных пар значений (a, b) инкрементируется счётчик в локальной хэш-таблице `pair_counts`. По завершении прохода в накопительную структуру шаблонов добавляются те пары, чьи счётчики не ниже значения `min_support_`.

Фаза 2 (`MinimalCover`). Накопленные шаблоны группируются по ключу `(lhs_attr_idx, lhs_value)`. Для каждой группы создаётся одна запись таблицы условий: значение левой части помещается в позицию своего атрибута, значения правой части — в позиции соответствующих атрибутов правой стороны. Если для того же ключа правая часть в данной позиции уже содержит значение, и оно отличается от нового, новое значение добавляется через запятую — так в строке шаблона возникает дизъюнкция.

Каждому шаблону присваивается значение поддержки, на основе которого вычисляются метрики валидности и полноты; полученные шаблоны помещаются в коллекцию `Condition` объекта `CIND`.

Параметры. Алгоритм `Cure` управляется единственным новым параметром — `support`, минимальным числом кортежей, подтверждающих шаблон (значение по умолчанию — 2). Параметр добавлен в систему опций `Desbordante`; внутри `CureCind` он хранится в поле `min_support_`. Выбор между алгоритмами поиска условий выполняется через расширенный `enum AlgoType` со значениями `cinderella`, `pli_cind` и `cure_cind`.

3.3. Алгоритм валидации

Класс `CINDVerifier` принимает на вход две таблицы, индексы атрибутов включения для каждой из них, а также набор значений условия. Поддерживаются оба типа условий: строковые (`row`) и групповые (`group`), задаваемые через параметр `condition_type`.

Алгоритм валидации работает в два этапа.

Этап 1. Построение множества включённых значений. Из правой таблицы I_2 извлекаются все уникальные комбинации значений атрибутов включения Y и помещаются в хэш-множество. Это позволяет за время $O(1)$ проверять, является ли конкретный кортеж из I_1 включённым.

Этап 2. Проверка кортежей левой таблицы. Кортежи из I_1 группируются по значениям атрибутов включения X . Для каждой группы определяется:

1. является ли группа включённой (присутствует ли значение x в хэш-множестве из этапа 1);
2. удовлетворяют ли кортежи группы заданному условию.

Если все значения условия являются подстановочными символами, валидатор использует упрощённый путь: проверяется только включённость каждой группы без фильтрации по условию. В противном случае для каждого кортежа выполняется сопоставление с условием: закодированное значение каждого атрибута условия сравнивается с ожидаемым значением, причём подстановочные символы пропускаются.

Группы, содержащие кортежи, удовлетворяющие условию, но не являющиеся включёнными, фиксируются как нарушающие. Для каждой такой группы создаётся объект `ViolatingCluster` с индексами всех строк группы и индексами строк-нарушителей.

По завершении обхода вычисляются метрики:

$$\text{validity} = \frac{\text{included_support}}{\text{supporting_baskets}}, \quad \text{completeness} = \frac{\text{included_support}}{\text{included_baskets_total}},$$

где `supporting_baskets` — число групп (или строк), удовлетворяющих условию, `included_support` — число таких групп, которые при этом являются включёнными, а `included_baskets_total` — общее число включённых групп.

Совместимость с алгоритмами поиска условий. Валидатор поддерживает формат условий, выдаваемый алгоритмами CINDERELLA и PLI-CIND — одно значение на каждый атрибут условия из левой таблицы. Алгоритм Cure формирует условия в более широком формате: помимо значений для атрибутов левой таблицы, он заполняет значения для атрибутов правой таблицы, а в фазе `MinimalCover` склеивает несколько значений одного атрибута через запятую (дизъюнкция). Поддержка такого формата требует расширения интерфейса `cind_condition_values` и логики сопоставления значений и в данной работе не реализована.

3.4. Интеграция в Desbordante

3.4.1. Пользовательские параметры

Валидатор `CINDVerifier` предоставляет следующие параметры, задаваемые через систему опций Desbordante:

- `tables` — список входных таблиц (минимум две: левая и правая);
- `lhs_indices` — индексы атрибутов включения в левой таблице;
- `rhs_indices` — индексы атрибутов включения в правой таблице;
- `cind_condition_values` — список значений условия, выровненный по порядку атрибутов условия (подстановочный символ "-" или "_" для произвольного значения);
- `condition_type` — тип условия: "row" для строковых или "group" для групповых метрик.

Атрибуты условия определяются автоматически как все столбцы левой таблицы, не входящие в `lhs_indices` (а при валидации в рамках одной таблицы — также не входящие в `rhs_indices`).

3.4.2. Python-привязки

Валидатор доступен из Python через модуль `cind_verification` пакета `desbordante`. Привязки реализованы с помощью библиотеки `ru-bind11` и предоставляют следующий интерфейс:

- `holds()` — возвращает `True`, если CIND выполняется с заданными порогами;
- `get_real_validity()` — вычисленная валидность;
- `get_real_completeness()` — вычисленная полнота;
- `get_violating_clusters()` — список объектов `ViolatingCluster`, каждый из которых содержит поля `basket_rows` и `violating_rows`;
- `get_violating_clusters_count()`, `get_violating_rows_count()` — количество нарушающих кластеров и строк.

В данном примере проверяется CIND на двух таблицах с атрибутом включения по первому столбцу и условием на четвёртый атрибут условия, равном `"Actor"`. Подстановочные символы `"_"` означают отсутствие ограничения на соответствующие атрибуты условия.

```

import desbordante
import pandas as pd

lhs_df = pd.read_csv("en.csv")
rhs_df = pd.read_csv("de.csv")

algo = desbordante.cind_verification.algorithms.Default()
algo.load_data(tables=[lhs_df, rhs_df])
algo.execute(
    lhs_indices=[0],
    rhs_indices=[0],
    condition_type="group",
    cind_condition_values=["_", "_", "_", "Actor"]
)

print(f"Holds: {algo.holds()}")
print(f"Validity: {algo.get_real_validity():.3f}")
print(f"Completeness: {algo.get_real_completeness():.3f}")

for cluster in algo.get_violating_clusters():
    print(f"Basket: {cluster.basket_rows}")
    print(f"Violations: {cluster.violating_rows}")

```

Рис. 10: Пример валидации CIND из Python

4. Эксперимент

Целью эксперимента является оценка влияния проведённых оптимизаций на производительность алгоритмов поиска CIND, а также измерение производительности реализованного валидатора. В рамках эксперимента были поставлены следующие вопросы:

1. Сохраняют ли проведённые оптимизации выдачу алгоритмов без изменений?
2. Какой выигрыш по времени работы и потреблению памяти дают оптимизации?
3. Как ведут себя три алгоритма поиска условий — CINDERELLA, PLI-CIND и Cure — при варьировании их параметров: каковы время работы и потребление памяти?
4. Каковы время работы и потребление памяти валидатора CIND по сравнению с полным поиском условий?

4.1. Набор данных и оборудование

Для проведения экспериментов были использованы пять наборов данных из [31] разного объёма и сложности. Во всех экспериментах метрика ошибки AIND зафиксирована на значении `error = 0,5`, а тип условий — групповой; этот выбор соответствует экспериментальной конфигурации из [31]. Указанное ниже число AIND для каждого набора получено именно при `error = 0,5`. Конкретные параметры алгоритмов поиска условий приводятся в соответствующих разделах.

1. **Brazilian E-Commerce**² — набор реальных данных о транзакциях (9 таблиц, 52 атрибута, 1,6М строк). Содержит 54 AIND, что создаёт высокую нагрузку на алгоритмы поиска условий;

²<https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>

2. **CIND DbPedia Example** — эталонный набор из [18] (2 таблицы, 10 атрибутов, 14 строк, 5 AIND). Используется для сверки выдачи с эталонными ответами из статьи;
3. **DbPedia En/De Persons**³ — данные о людях из английской и немецкой версий DBpedia (2 таблицы, 21 атрибут, 1,46М строк, 22 AIND). Большое число AIND при значительном объёме данных делает этот набор одним из наиболее требовательных;
4. **Cartoon/Comics Titles**⁴ — компактный набор (2 таблицы, 18 атрибутов, 90К строк, 3 AIND), удобный для детального сравнения алгоритмов;
5. **US Baby Names**⁵ — две таблицы с более чем 3,6М строк каждая (10 атрибутов, 4 AIND). Позволяет оценить масштабируемость алгоритмов на большом объёме данных.

Эксперименты проводились на следующем оборудовании:

- ОС: Ubuntu 24.04 LTS;
- Процессор: Intel Core i5-14400F, 10 ядер, 0,8/4,7 ГГц;
- Оперативная память: 64 ГБ;
- Хранилище: NVMe SSD, 1 ТБ;
- Компилятор: GCC 13.3.0, boost 1.90.0, уровень оптимизации -O3.

Каждый замер повторялся 20 раз, в результатах приведены средние значения и стандартные отклонения в формате $\bar{t} \pm \sigma$. Для пикового потребления памяти отклонение пренебрежимо мало (менее 0,1%) и не приводится отдельно.

³<https://downloads.dbpedia.org/wiki-archive/data-set-20.html>

⁴<https://www.kaggle.com/datasets/nikhil1e9/myanimelist-anime-and-manga>

⁵<https://www.kaggle.com/datasets/robikscube/us-baby-name-popularity>

4.2. Эквивалентность выдачи оптимизированных алгоритмов

Чтобы убедиться, что оптимизации не изменили выдачу, оптимизированные версии CINDERELLA и PLI-CIND были запущены на всех пяти наборах данных при $\text{validity} = 0,9$ и $\text{completeness} = 0,05$, и их выдача сравнивалась с выдачей исходных реализаций при тех же параметрах. Для каждой обнаруженной AIND сравнивались найденные условия, а также значения метрик валидности и полноты.

Во всех случаях результаты оптимизированных версий полностью совпали с результатами до оптимизации: множества найденных условий идентичны, метрики валидности и полноты совпадают с точностью до 10^{-9} , порядок обработки AIND и число итераций не изменились. Это подтверждает, что проведённые оптимизации затрагивают только внутреннее представление данных и не влияют на результаты работы алгоритмов.

4.3. Влияние оптимизаций на производительность

Для оценки влияния оптимизаций были измерены время работы и пиковое потребление оперативной памяти исходных и оптимизированных версий алгоритмов. Замеры выполнены при $\text{validity} = 0,9$ и $\text{completeness} = 0,05$ — значениях, использованных в [31]. Набор CIND DbPedia Example (14 строк) слишком мал для замеров производительности, поэтому результаты для него не приводятся. Данные для остальных четырёх наборов представлены в табл. 1 и 2.

Таблица 1: Сравнение времени работы (секунды), групповые условия

Набор данных	CINDERELLA			PLI-CIND		
	Исх.	Опт.	Ускор.	Исх.	Опт.	Ускор.
Cartoon/Comics	$1,42 \pm 0,03$	$0,79 \pm 0,02$	1,8×	$2,87 \pm 0,07$	$1,53 \pm 0,04$	1,9×
US Baby Names	$3,84 \pm 0,09$	$1,87 \pm 0,05$	2,0×	$8,72 \pm 0,21$	$3,93 \pm 0,11$	2,2×
Brazilian E-Comm.	—	$14,28 \pm 0,38$	—	$19,12 \pm 0,54$	$8,24 \pm 0,19$	2,3×
DbPedia En/De	—	$17,58 \pm 0,41$	—	$32,37 \pm 0,95$	$13,82 \pm 0,31$	2,3×

Оба алгоритма работают быстрее, причём ускорение плавно растёт с увеличением объёма данных и числа AIND: от 1,8–1,9 раза на компакт-

Таблица 2: Сравнение потребления памяти (МБ), групповые условия

Набор данных	CINDERELLA			PLI-CIND		
	Исх.	Опт.	Сниж.	Исх.	Опт.	Сниж.
Cartoon/Comics	120	52	2,3×	34	28	1,2×
US Baby Names	980	350	2,8×	78	62	1,3×
Brazilian E-Comm.	>65536	2800	—	185	145	1,3×
DbPedia En/De	>65536	3400	—	220	170	1,3×

ном наборе Cartoon/Comics до 2,3 раза на больших наборах Brazilian E-Commerce и DbPedia En/De. Замена упорядоченных контейнеров на хэш-контейнеры даёт тем больший эффект, чем больше уникальных значений хранится в контейнерах.

PLI-CIND получил несколько большее ускорение, чем CINDERELLA. Это объясняется тем, что PLI-CIND при классификации строк использует строковые ключи, и переход от логарифмического к амортизированно константному поиску даёт на строковых сравнениях больший выигрыш, чем на целочисленных.

Потребление памяти CINDERELLA сократилось в 2,3–2,8 раза. Исправление утечки памяти, вызванной циклическими ссылками в дереве наборов элементов, позволило CINDERELLA впервые завершить работу на наборах Brazilian E-Commerce и DbPedia En/De — ранее исходная реализация превышала лимит оперативной памяти (64 ГБ). На наборе US Baby Names CINDERELLA укладывается в лимит памяти, несмотря на большой объём (3,6 М строк): из-за низкой кардинальности данных (имена часто повторяются) число групп в групповом режиме на порядки меньше числа строк, что существенно сокращает размер внутренних структур.

У PLI-CIND потребление памяти снизилось незначительно (в 1,2–1,3 раза), что объясняется изначально умеренным и предсказуемым расходом памяти этого алгоритма.

4.4. Сравнение алгоритмов поиска условий

Для сопоставления трёх алгоритмов поиска условий — CINDERELLA, PLI-CIND и Cure — был проведён эксперимент с фиксированным

набором AIND и варьированием параметров каждого алгоритма. Метрика ошибки AIND зафиксирована на значении **error** = 0,5, что обеспечивает идентичный набор входных AIND для всех трёх алгоритмов на каждом наборе данных. Использовались оптимизированные версии CINDERELLA и PLI-CIND, тип условий — групповой.

Варьировались параметры, оказывающие основное влияние на размер пространства кандидатов:

- для CINDERELLA и PLI-CIND — параметр **completeness** при фиксированном **validity** = 0,9:

$$\text{completeness} \in \{0,01; 0,05; 0,1; 0,2; 0,5\}.$$

Именно **completeness** позволяет отсекать неперспективные ветви на ранних уровнях обхода;

- для Cure — единственный параметр алгоритма **support**:

$$\text{support} \in \{2, 5, 10, 50, 200\}.$$

На каждой комбинации параметров измерялись время работы и пиковое потребление оперативной памяти.

Эксперимент проводился на трёх наборах данных с разным сочетанием объёма и числа AIND:

- Brazilian E-Commerce (1,6 М строк, 54 AIND) — наибольшая нагрузка за счёт максимального числа AIND;
- DbPedia En/De (1,46 М строк, 22 AIND) — большой объём данных при заметно меньшем числе AIND;
- Cartoon/Comics (90 К строк, 3 AIND) — небольшой набор с малой нагрузкой.

Набор US Baby Names не использовался: при объёме 3,6 М строк имена в нём многократно повторяются, поэтому в групповом режиме число

корзин на порядки меньше числа строк, и по фактической нагрузке набор оказывается ближе к Cartoon/Comics.

Результаты для каждого набора данных приведены в табл. 3–5. Во всех таблицах значения параметров упорядочены от наименее к наиболее строгим.

Таблица 3: Сравнение алгоритмов на наборе Brazilian E-Commerce

Алгоритм	Параметр	Время, с	Память, МБ	#CIND
CINDERELLA	completeness = 0,01	$23,47 \pm 0,65$	4150	54
	completeness = 0,05	$14,28 \pm 0,38$	2800	52
	completeness = 0,1	$9,68 \pm 0,27$	2050	47
	completeness = 0,2	$5,72 \pm 0,16$	1450	34
	completeness = 0,5	$2,63 \pm 0,07$	850	16
PLI-CIND	completeness = 0,01	$12,87 \pm 0,36$	152	54
	completeness = 0,05	$8,24 \pm 0,19$	145	52
	completeness = 0,1	$5,58 \pm 0,16$	143	47
	completeness = 0,2	$3,62 \pm 0,10$	142	34
	completeness = 0,5	$1,72 \pm 0,05$	140	16
Cure	support = 2	$11,42 \pm 0,31$	472	54
	support = 5	$11,18 \pm 0,30$	468	53
	support = 10	$10,97 \pm 0,29$	464	50
	support = 50	$10,82 \pm 0,28$	460	42
	support = 200	$10,63 \pm 0,27$	456	28

Таблица 4: Сравнение алгоритмов на наборе DbPedia En/De

Алгоритм	Параметр	Время, с	Память, МБ	#CIND
CINDERELLA	completeness = 0,01	$28,68 \pm 0,79$	4900	22
	completeness = 0,05	$17,58 \pm 0,41$	3400	22
	completeness = 0,1	$11,82 \pm 0,31$	2500	20
	completeness = 0,2	$6,97 \pm 0,20$	1750	16
	completeness = 0,5	$3,23 \pm 0,09$	1050	8
PLI-CIND	completeness = 0,01	$21,47 \pm 0,58$	178	22
	completeness = 0,05	$13,82 \pm 0,31$	170	22
	completeness = 0,1	$9,52 \pm 0,26$	168	20
	completeness = 0,2	$5,87 \pm 0,16$	165	16
	completeness = 0,5	$2,82 \pm 0,07$	162	8
Cure	support = 2	$17,62 \pm 0,46$	582	22
	support = 5	$17,38 \pm 0,45$	578	22
	support = 10	$17,17 \pm 0,44$	575	21
	support = 50	$16,92 \pm 0,43$	570	18
	support = 200	$16,57 \pm 0,42$	565	12

Влияние параметра на время работы. При переходе от completeness = 0,01 к 0,5 время работы CINDERELLA и PLI-CIND сокра-

Таблица 5: Сравнение алгоритмов на наборе Cartoon/Comics

Алгоритм	Параметр	Время, с	Память, МБ	#CIND
CINDERELLA	<code>completeness = 0,01</code>	$1,32 \pm 0,04$	76	3
	<code>completeness = 0,05</code>	$0,82 \pm 0,02$	52	3
	<code>completeness = 0,1</code>	$0,55 \pm 0,02$	41	3
	<code>completeness = 0,2</code>	$0,33 \pm 0,01$	32	3
	<code>completeness = 0,5</code>	$0,16 \pm 0,01$	22	2
PLI-CIND	<code>completeness = 0,01</code>	$2,43 \pm 0,07$	30	3
	<code>completeness = 0,05</code>	$1,53 \pm 0,04$	28	3
	<code>completeness = 0,1</code>	$1,05 \pm 0,03$	27	3
	<code>completeness = 0,2</code>	$0,66 \pm 0,02$	26	3
	<code>completeness = 0,5</code>	$0,31 \pm 0,01$	25	2
Cure	<code>support = 2</code>	$1,05 \pm 0,03$	56	3
	<code>support = 5</code>	$1,04 \pm 0,03$	54	3
	<code>support = 10</code>	$1,02 \pm 0,03$	52	3
	<code>support = 50</code>	$1,01 \pm 0,03$	50	3
	<code>support = 200</code>	$0,98 \pm 0,03$	48	2

щается в 8–9 раз: чем выше порог, тем меньше кандидатов доходит до полной проверки. Время работы Cure при изменении `support` в 100 раз меняется лишь на 6–8%, поскольку соединение таблиц и подсчёт пар значений выполняются до применения `support`, а сам параметр служит только порогом фильтрации уже построенных шаблонов.

Потребление памяти. Зависимость потребления памяти от параметра у каждого алгоритма различна. CINDERELLA хранит хэш-таблицы кандидатов всех уровней обхода, поэтому при низком `completeness` расход памяти существенно возрастает: на больших наборах он растёт от 850–1050 МБ при `completeness = 0,5` до 4–5 ГБ при 0,01. PLI-CIND, использующий обход в глубину, удерживает в памяти только текущую ветку рекурсии; разброс потребления памяти по диапазону `completeness` не превышает 10%, на больших наборах PLI-CIND укладывается в 140–180 МБ. У Cure расход памяти также практически не зависит от `support` и определяется хэш-таблицами для соединения таблиц R_1 и R_2 : на больших наборах он составляет 460–580 МБ, что превышает потребление PLI-CIND, но существенно меньше потребления CINDERELLA.

Выбор алгоритма. На больших наборах после оптимизаций PLI-CIND превосходит CINDERELLA по обеим характеристикам: он рабо-

тает в 1,3–1,8 раза быстрее и расходует в 15–20 раз меньше памяти. На компактных наборах быстрее CINDERELLA (в 1,5–2 раза), а разрыв в потреблении памяти невелик и при строгом **completeness** инвертируется в её пользу. Sure целесообразно применять при необходимости компактного представления условий с дизъюнкциями — например, для последующей очистки данных, — а также в случаях, когда задание абсолютного порога поддержки естественнее относительных метрик **validity** и **completeness**.

4.5. Производительность валидатора

Для оценки производительности валидатора CIND были проведены замеры времени работы и пикового потребления памяти на тех же четырёх наборах данных. Для каждого набора валидатор запускался в двух режимах: с пустым условием (все подстановочные символы, что эквивалентно проверке обычной IND) и с конкретным условием — одним из условий, найденных алгоритмом CINDERELLA на этом же наборе. Во всех экспериментах использовался групповой тип условий.

Таблица 6: Время работы валидатора (секунды), групповые условия

Набор данных	Все wildcard	Конкретное условие
Cartoon/Comics	$0,32 \pm 0,01$	$0,43 \pm 0,01$
US Baby Names	$0,87 \pm 0,02$	$1,13 \pm 0,03$
Brazilian E-Comm.	$1,77 \pm 0,05$	$2,14 \pm 0,06$
DbPedia En/De	$1,52 \pm 0,04$	$1,93 \pm 0,05$

Таблица 7: Потребление памяти валидатора (МБ), групповые условия

Набор данных	Все wildcard	Конкретное условие
Cartoon/Comics	18	24
US Baby Names	45	58
Brazilian E-Comm.	82	95
DbPedia En/De	74	88

Валидатор работает существенно быстрее полного поиска условий: от 2 раз на компактном наборе Cartoon/Comics до 7–9 раз на больших наборах по сравнению с оптимизированными версиями обоих майнеров. Это объясняется тем, что валидатор выполняет один проход по данным

для проверки конкретного условия, тогда как майнеры перебирают всё пространство кандидатов.

Режим с пустым условием работает быстрее и потребляет меньше памяти, чем режим с конкретным условием. Это объясняется тем, что при пустом условии валидатор использует упрощённый путь: работает напрямую со строковыми потоками, не выполняя кодирование таблиц. При задании конкретного условия требуется построение закодированного представления таблиц и сопоставление каждого кортежа с условием, что увеличивает как время работы, так и потребление памяти.

Потребление памяти валидатора остаётся умеренным во всех случаях и линейно зависит от объёма входных данных, не превышая 95 МБ даже на больших наборах.

Заключение

В результате проделанной работы были выполнены следующие задачи:

1. Оптимизированы реализации алгоритмов поиска условных зависимостей включения CINDERELLA и PLI-CIND: упорядоченные контейнеры заменены на хэш-контейнеры, связные списки — на непрерывные массивы для улучшения кэш-локальности, устранена утечка памяти в дереве наборов элементов CINDERELLA;
2. Разработан и интегрирован в Desbordante алгоритм валидации CIND, проверяющий выполнимость заданной зависимости на конкретных данных, с Python-привязками, тестами и примерами использования;
3. Реализован интегрирован в Desbordante альтернативный алгоритм поиска CIND — Cure, с тестами и примерами использования;
4. Проведено экспериментальное исследование на пяти наборах данных. Оптимизации не изменили выдачу алгоритмов и дали ускорение в 1,8–2,3 раза, а потребление памяти CINDERELLA снизилось в 2,3–2,8 раза, что позволило алгоритму завершить работу на Brazilian E-Commerce и DbPedia En/De, ранее недоступных из-за нехватки памяти. Сравнение трёх алгоритмов показало, что на крупных наборах PLI-CIND превосходит CINDERELLA по обеим характеристикам, а на компактных — наоборот; Cure стабилен по своему параметру. Валидатор CIND работает в 2–9 раз быстрее полного поиска при умеренном расходе памяти.

Реализация доступна в виде двух pull request'ов в репозитории Desbordante: PR #664⁶ (оптимизации, валидатор, привязки) и PR #729⁷ (Cure).

⁶<https://github.com/Desbordante/desbordante-core/pull/664>

⁷<https://github.com/Desbordante/desbordante-core/pull/729>

Список литературы

- [1] Abedjan Ziawasch, Golab Lukasz, Naumann Felix. Profiling relational data: a survey // [The VLDB Journal](#). — 2015. — Vol. 24, no. 4. — P. 557–581.
- [2] Abiteboul Serge, Hull Richard, Vianu Victor. Foundations of Databases. — Addison-Wesley, 1995. — ISBN: [0-201-53771-0](#).
- [3] Agrawal Rakesh, Srikant Ramakrishnan. Fast Algorithms for Mining Association Rules in Large Databases // Proceedings of the 20th International Conference on Very Large Data Bases. — VLDB '94. — San Francisco, CA, USA : Morgan Kaufmann Publishers Inc., 1994. — P. 487–499.
- [4] Bleifuss Tobias, Papenbrock Thorsten, Naumann Felix. [Discovering Functional Dependencies through Hitting Set Enumeration](#). — Vol. 2. — 2024. — P. 1–26.
- [5] Bravo Loreto, Fan Wenfei, Ma Shuai. Extending Dependencies with Conditions // Proceedings of the 33rd International Conference on Very Large Data Bases. — VLDB '07. — VLDB Endowment, 2007. — P. 243–254.
- [6] Casanova Marco A., Fagin Ronald, Papadimitriou Christos H. Inclusion Dependencies and Their Interaction with Functional Dependencies // [Journal of Computer and System Sciences](#). — 1984. — Vol. 28, no. 1. — P. 29–59.
- [7] Chaudhuri Surajit. [An Overview of Query Optimization in Relational Systems](#) // Proceedings of the Seventeenth ACM SIGACT-SIGMOD-SIGART Symposium on Principles of Database Systems. — PODS '98. — ACM, 1998. — P. 34–43.
- [8] Chiang Fei, Miller Renée J. Discovering Data Quality Rules // [Proceedings of the VLDB Endowment](#). — 2008. — Vol. 1, no. 1. — P. 1166–1177.

- [9] Codd E. F. A Relational Model of Data for Large Shared Data Banks // [Communications of the ACM](#). — 1970. — Vol. 13, no. 6. — P. 377–387.
- [10] [Conditional Functional Dependencies for Data Cleaning](#) / Philip Bohannon, Wenfei Fan, Floris Geerts et al. // 2007 IEEE 23rd International Conference on Data Engineering. — IEEE, 2007. — P. 746–755.
- [11] Curé Olivier. Improving the Data Quality of Drug Databases using Conditional Dependencies and Ontologies // [ACM Journal of Data and Information Quality](#). — 2012. — Vol. 4, no. 1. — P. 20:1–20:23.
- [12] Data Profiling / Ziawasch Abedjan, Lukasz Golab, Felix Naumann, Thorsten Papenbrock. Synthesis Lectures on Data Management. — Morgan & Claypool Publishers, 2018. — ISBN: [9781681734460](#).
- [13] Data Profiling with Metanome / Thorsten Papenbrock, Tanja Bergmann, Moritz Finke et al. // [Proceedings of the VLDB Endowment](#). — 2015. — Vol. 8, no. 12. — P. 1860–1863.
- [14] Davoudian Ali, Chen Liu, Liu Mengchi. A Survey on NoSQL Stores // [ACM Computing Surveys](#). — 2018. — Vol. 51, no. 2. — P. 40:1–40:43.
- [15] De Marchi Fabien, Lopes Stéphane, Petit Jean-Marc. Unary and n-ary Inclusion Dependency Discovery in Relational Databases // [Journal of Intelligent Information Systems](#). — 2009. — Vol. 32, no. 1. — P. 53–73.
- [16] [Desbordante: from Benchmarking Suite to High-Performance Science-Intensive Data Profiler](#) / George Chernishev, Michael Polyntsov, Anton Chizhov et al. // Proceedings of the 8th International Conference on Data Science and Management of Data (12th ACM IKDD CODS and 30th COMAD). — ACM, 2025.
- [17] Discover Dependencies from Data—A Review / Jixue Liu, Jiuyong Li, Chengfei Liu, Yongfeng Chen // [IEEE Transactions on Knowledge and Data Engineering](#). — 2012. — Vol. 24, no. 2. — P. 251–264.

- [18] [Discovering Conditional Inclusion Dependencies](#) / Jana Bauckmann, Ziawasch Abedjan, Ulf Leser et al. // Proceedings of the 21st ACM International Conference on Information and Knowledge Management. — CIKM '12. — ACM, 2012. — P. 2094–2098.
- [19] Discovering Functional and Inclusion Dependencies in Relational Databases / Martti Kantola, Heikki Mannila, Kari-Jouko Räihä, Harri Siirtola // [International Journal of Intelligent Systems](#). — 1992. — Vol. 7, no. 7. — P. 591–607.
- [20] Divide & Conquer-based Inclusion Dependency Discovery / Thorsten Papenbrock, Sebastian Kruse, Jorge-Arnulfo Quiané-Ruiz, Felix Naumann // [Proceedings of the VLDB Endowment](#). — 2015. — Vol. 8, no. 7. — P. 774–785.
- [21] [Efficiently Detecting Inclusion Dependencies](#) / Jana Bauckmann, Ulf Leser, Felix Naumann, Veronique Tietz // 2007 IEEE 23rd International Conference on Data Engineering. — IEEE, 2007. — P. 1448–1450.
- [22] Fast Approximate Discovery of Inclusion Dependencies / Sebastian Kruse, Thorsten Papenbrock, Christian Dullweber et al. // Datenbanksysteme für Business, Technologie und Web (BTW 2017). — Vol. P-265 of LNI. — GI, 2017. — P. 207–226.
- [23] [Inclusion Dependency Discovery: An Experimental Evaluation of Thirteen Algorithms](#) / Falco Dürsch, Axel Stebner, Fabian Windheuser et al. // Proceedings of the 28th ACM International Conference on Information and Knowledge Management. — CIKM '19. — ACM, 2019. — P. 219–228.
- [24] Levene Mark, Vincent Millist W. Justification for Inclusion Dependency Normal Form // [IEEE Transactions on Knowledge and Data Engineering](#). — 2000. — Vol. 12, no. 2. — P. 281–291.
- [25] A Machine Learning Approach to Foreign Key Discovery / Alexandra Rostin, Oliver Albrecht, Jana Bauckmann et al. // Proceed-

ings of the 12th International Workshop on the Web and Databases (WebDB). — 2009.

- [26] Naumann Felix. Data Profiling Revisited // [SIGMOD Record](#). — 2014. — Vol. 42, no. 4. — P. 40–49.
- [27] On Generating Near-Optimal Tableaux for Conditional Functional Dependencies / Lukasz Golab, Howard Karloff, Flip Korn et al. // [Proceedings of the VLDB Endowment](#). — 2008. — Vol. 1, no. 1. — P. 376–390.
- [28] Papenbrock Thorsten, Naumann Felix. [A Hybrid Approach to Functional Dependency Discovery](#) // Proceedings of the 2016 International Conference on Management of Data. — SIGMOD '16. — ACM, 2016. — P. 821–833.
- [29] Rahm Erhard, Do Hong Hai. Data Cleaning: Problems and Current Approaches // IEEE Data Engineering Bulletin. — 2000. — Vol. 23, no. 4. — P. 3–13.
- [30] TANE: An Efficient Algorithm for Discovering Functional and Approximate Dependencies / Ykä Huhtala, Juha Kärkkäinen, Pasi Porkka, Hannu Toivonen // [The Computer Journal](#). — 1999. — Vol. 42, no. 2. — P. 100–111.
- [31] Абросимов Григорий. [Реализация алгоритмов поиска условных зависимостей включения в инструменте профилирования данных Desbordante](#). — 2025.